Guía de estudio – Parcial de Criptografía y AppSec

Oct 14, 2026 · @kevin

El parcial te da un caso y evalúa si eliges la herramienta correcta en cada punto (hash, AES, RSA) y si justificas por qué; esta guía te da el criterio, un esquema robusto tipo Bitwarden y casos resueltos.

Cómo atacar un caso

Todo caso se resuelve igual: qué proteges, de quién, qué propiedad necesitas y qué herramienta la da. Sigue estos pasos en orden y justifica cada uno.

1. Lista los activos (contraseñas, archivos, mensajes) y si están en tránsito o en reposo.
2. Define al atacante: quien escucha la red (Eve), quien se hace pasar por otro (man-in-the-middle), quien roba la base de datos, o el propio proveedor.
3. Nombra la propiedad que falta: confidencialidad, integridad, autenticación (¿quién eres?) o no repudio.
4. Asigna una herramienta por propiedad (tabla de la sección siguiente).
5. Resuelve la llave: ¿de dónde sale, quién la conoce, dónde se guarda? Es el problema central del capítulo 1.
6. Busca el eslabón débil: modo de operación, relleno, dónde queda la llave, implementación. Un eslabón débil rompe toda la cadena.

Qué problema resuelve cada herramienta

Cada herramienta cubre una propiedad y deja huecos; por eso los sistemas reales las combinan.

Herramienta	Resuelve	No resuelve	Uso típico
Hash (SHA-256)	Huella de tamaño fijo, determinista, unidireccional; un bit cambiado cambia todo (integridad)	No cifra y no autentica: cualquiera puede recalcularlo	Verificar descargas, detectar manipulación, firmar sin exponer el mensaje

Herramienta	Resuelve	No resuelve	Uso típico
AES (simétrico)	Confidencialidad rápida de datos masivos; bloque de 128 bits, llave de 128/192/256	Cómo entregar la llave; el modo importa (ECB filtra patrones, CBC usa IV aleatorio)	Cifrar el baúl, discos, sesiones TLS
RSA (asimétrico)	Cifrar con la llave pública del destinatario sin secreto previo; firmar con tu privada	Es lento y costoso; exige relleno (padding); no prueba identidad sin una firma o certificado	Transportar llaves, firmar
Diffie–Hellman	Acordar una llave compartida sin enviarla	No cifra nada y no autentica: sufre man-in-the-middle	Intercambio de llaves antes de AES
Firma digital	Identidad, integridad y no repudio	No da confidencialidad por sí sola	Contratos, software, mensajes
KDF (PBKDF2)	Convierte una contraseña humana en llave y la hace costosa de adivinar	No hace fuerte una contraseña débil	Derivar el password maestro

Dos reglas de las diapositivas: cifra con la llave pública del destinatario y firma con tu propia llave privada; y firma el hash, nunca el mensaje. La fila del KDF no está en las diapositivas; es lo que tu profe pide con "derivar un password maestro".

Password maestro: derivar para autenticar y para cifrar

De un solo password maestro salen dos secretos distintos: uno que se envía al servidor para autenticarte y otro que nunca sale de tu dispositivo y cifra el baúl. Así funciona Bitwarden según su documentación pública (no está en las diapositivas; confirma los detalles con tus apuntes de clase).

```
password maestro + email (sal)
        |  PBKDF2-SHA256, N iteraciones
        v
  Master Key (256 bits)  -- nunca sale del dispositivo
     |                          |
     | 1 iteración más           | HKDF (expansión)
     | con el password           v
     v                    Stretched Master Key
 Master Password Hash           | cifra con AES-256
 (AUTENTICACIÓN)                v
     |               Protected Symmetric Key
     v               (llave aleatoria de la cuenta, guardada cifrada)
 servidor: lo vuelve a hashear con sal y guarda
```

- Autenticación: el servidor solo ve el Master Password Hash y lo compara. Como el hash es unidireccional, de ahí no se recupera la Master Key.
- Cifrado: la llave que protege los datos es una llave simétrica aleatoria de la cuenta, guardada cifrada con la Stretched Master Key. Cambiar el password solo re-cifra esa llave, no todo el baúl.
- Los elementos del baúl se cifran con AES-256 más un código de autenticación (HMAC) para detectar manipulación.
- Compartir: un par RSA por usuario; la llave de una organización se cifra con la llave pública de cada miembro.
- Autenticación vs autorización: autenticar es probar quién eres (el hash del password maestro); autorizar es decidir qué puedes hacer (a qué baúles u organizaciones accedes), y en el esquema cifrado se refuerza criptográficamente: solo quien recibió la llave de la organización puede descifrar sus datos.

Cifrado en reposo y zero knowledge

Zero knowledge significa que el proveedor guarda tus datos pero no tiene ninguna forma de leerlos: todo se cifra en tu dispositivo antes de subirlo.

Por qué no pueden abrir tu baúl:

1. La llave de cifrado se deriva de tu password en tu dispositivo y nunca viaja.
2. Lo que viaja para autenticarte es otro valor, derivado con un paso unidireccional más; de él no se puede volver a la llave.
3. En el servidor solo hay datos cifrados con AES, la llave de cuenta cifrada y el hash de autenticación.
4. Kerckhoffs: todo el sistema puede ser público; solo la llave es secreta.

Cómo se combinan las tres categorías: el hash/KDF separa autenticación de cifrado; AES protege los datos con una llave que el servidor nunca ve; RSA permite compartir llaves sin secreto previo. Ninguna basta sola: el hash no cifra, AES no resuelve la distribución de la llave y RSA es demasiado lento para datos masivos (por eso el patrón híbrido: asimétrico para la llave, simétrico para el contenido).

Límites que conviene nombrar en el parcial: un password débil permite ataque de fuerza bruta offline; si pierdes el password maestro pierdes el baúl (como una llave privada perdida); y el cliente que descarga el usuario debe ser confiable.

Iteraciones y complejidad

Las iteraciones hacen que cada intento de adivinar el password cueste N veces más; el usuario legítimo paga ese costo una vez, el atacante lo paga en cada intento. Un hash normal es rápido a propósito (sirve para integridad de gigabytes), por eso es mala idea guardar o derivar passwords con un solo SHA.

Ejemplo ilustrativo (supuestos míos, no de las diapositivas): un atacante con 1.000 millones de hashes por segundo y un password con 2^40 posibilidades.

Iteraciones	Intentos por segundo	Tiempo para recorrer 2^40
1	1.000.000.000	unos 18 minutos
200	5.000.000	unos 2,5 días
600.000	unos 1.700	unos 21 años

Sobre "por qué 200": no sé con exactitud qué cifra usó tu profe ni en qué contexto, así que pregúntale o revisa tus apuntes. El argumento que te sirve es el de la tabla: el costo crece linealmente con las iteraciones, y la cifra se elige para que el login siga siendo rápido pero la fuerza bruta sea inviable. Con 200 el margen es pequeño; recomendaciones recientes (OWASP, Bitwarden) hablan de cientos de miles para PBKDF2-SHA256, así que verifica la cifra vigente antes de citarla.

Complejidad que sí ayuda: sal única por usuario (evita tablas precalculadas) y, en el cap. 2, el tamaño de la llave (el DES de 56 bits cayó en 22 horas en 1999: el diseño aguantó, el espacio de llaves no). Complejidad que daña: inventar tu propio esquema; "la complejidad es enemiga de la seguridad" (cap. 1).

Cómo responder un caso

El profe pidió una solución robusta con un nivel de detalle razonable: nombra algoritmo, tamaño, modo y origen de cada llave, y explica qué propiedad da cada pieza. No escribas los pasos internos de AES o SHA.

Plantilla para cada flujo del caso:

1. Qué dato y qué atacante.
2. Qué propiedad se necesita.
3. Herramienta con parámetros (por ejemplo AES-256 con IV aleatorio y MAC; RSA-2048 o más con relleno; SHA-256; PBKDF2 con sal y muchas iteraciones).
4. De dónde sale la llave, quién la tiene y dónde se guarda.
5. Qué ve el atacante si intercepta o roba el servidor.
6. Riesgo residual y cómo se mitiga.

Trabaja con librerías probadas: las diapositivas insisten en no implementar cifrados ni hashes propios.

Caso resuelto: historias clínicas cifradas

Caso (inventado para practicar): una clínica guarda historias clínicas en la nube; el proveedor no debe poder leerlas y los médicos deben poder compartirlas entre sí.

1. Registro y login: el médico elige un password maestro. Se deriva una Master Key con PBKDF2-SHA256, sal única y muchas iteraciones. Un segundo valor derivado (hash de autenticación) viaja al servidor, que lo vuelve a hashear.
2. Cifrado en reposo: cada historia se cifra en el cliente con AES-256 y un IV aleatorio, más un MAC. La llave de cuenta aleatoria se guarda cifrada con la llave derivada del password.
3. Compartir: cada médico tiene un par RSA. Para compartir, la llave de la historia se cifra con la llave pública del colega (RSA con relleno); el contenido sigue cifrado con AES (esquema híbrido).
4. Identidad de las llaves públicas: si no se verifican, un atacante intermedio entrega la suya (lección del caso de los diez mil dólares). Se resuelve firmando o certificando las llaves públicas.
5. Integridad y no repudio: cada anotación se firma con la privada del médico, firmando el hash SHA-256 y no el texto.
6. En tránsito: TLS, que combina intercambio de llaves asimétrico y AES.

7. Qué ve un atacante que roba el servidor: datos cifrados, llaves cifradas y hashes de autenticación; para avanzar debe adivinar passwords, y cada intento cuesta N iteraciones.
8. Riesgo residual: password débil, llave privada filtrada en un repositorio o correo, pérdida del password. Mitigación: política de passwords, detección de secretos en el repositorio y un mecanismo de recuperación acordado.

Casos de práctica

Intenta cada uno por escrito con la plantilla antes de mirar la pista.

Caso	Pista de solución
Una empresa guarda las contraseñas de sus usuarios en su base de datos. ¿Cifrar o hashear?	Hashear con un KDF lento (PBKDF2), sal única e iteraciones: nadie debe poder recuperar el password, solo verificarlo. Un hash rápido simple facilita la fuerza bruta.
Publicas un instalador y un hash SHA-256 en la misma web. ¿Basta?	No: quien cambie el archivo cambia también el hash. Hay que firmar el hash con la llave privada del editor; el usuario verifica con la pública.
Dos desconocidos quieren intercambiar un documento por internet.	Diffie–Hellman para acordar la llave y AES para el documento; pero DH no autentica, así que un intermedio puede suplantar. Se añaden firmas (caso de los 10.000 dólares: el descifrado funcionó y el dinero fue a Eve).
Cifras imágenes con AES y se nota la silueta.	Modo ECB: bloques iguales dan cifrado igual. Usar CBC con IV aleatorio (o un modo autenticado).
Votación electrónica: el voto debe validarse sin saber quién vota.	Firma ciega (Chaum): el votante enmascara, la autoridad firma sin ver, el votante quita la máscara y queda con firma válida. Riesgo: el firmante no sabe qué firma, así que hay que limitar qué clase de documento es elegible.
Un sensor enterrado envía datos a dos países desconfiados (caso del tratado).	Firmar cada lectura con la privada guardada dentro del sensor; ambos verifican con la pública: uno obtiene transparencia, el otro integridad.

Trampas típicas y lo que enseñan las diapositivas

De los capítulos 1 a 3:

- Firma el hash, nunca el mensaje: firmar el texto lo publica, porque cualquiera con la pública lo recupera.
- Rellena (padding) los mensajes en RSA: un mensaje corto con exponente pequeño se deshace con aritmética normal; el relleno aleatorio también evita cifrados repetidos.
- Diffie–Hellman sin autenticación es vulnerable a man-in-the-middle.
- ECB filtra patrones; CBC usa IV aleatorio.
- La llave tiene vida útil: DES (56 bits) cayó en 22 horas en 1999; 3DES da 168 bits en papel pero unos 112 reales y ya está en desuso; AES es el reemplazo (2001).
- MD5 y SHA-1 están retirados (SHA-1 inseguro desde 2005); usa SHA-2 o SHA-3.
- Los ataques reales suelen ser de implementación (tiempo, consumo, caché), no matemáticos: usa librerías probadas.
- Una llave privada copiada a un laptop, enviada por correo o dejada en un repositorio es el fallo más común; una exposición parcial cuenta como total.
- ROT13 es ofuscación, no cifrado; Vernam es perfecto en teoría pero impráctico (llave aleatoria, tan larga como el mensaje, nueva en cada uso).
- Kerckhoffs: el sistema debe ser seguro aunque todo sea público salvo la llave; seguridad perfecta no es lo mismo que segura.

De AppSec part 1 (defensa en profundidad y "empujar a la izquierda", es decir, hacer seguridad desde las primeras fases):

Fase del SDLC	Actividades de seguridad
Requisitos	Historias de usuario de seguridad, aprobación de la tecnología propuesta
Diseño	Modelado de amenazas, revisión de la arquitectura
Código	SAST, SCA, detección de secretos, librerías seguras
Pruebas	DAST, pruebas de penetración, revisión de código
Despliegue	WAF, monitoreo y logging, respuesta a incidentes

Autoevaluación y fuentes

Si puedes responder esto sin mirar, estás listo:

- [ ] ¿Por qué AES solo no resuelve el problema de las llaves?

- [ ] ¿Qué propiedad da un hash y cuál no?
- [ ] ¿Por qué se firma el hash y no el mensaje?
- [ ] ¿Qué se envía al servidor para autenticarte y por qué no sirve para descifrar el baúl?
- [ ] ¿Por qué el proveedor no puede abrir tu baúl?
- [ ] ¿Cómo cambia el costo del atacante al subir las iteraciones?
- [ ] ¿Por qué Diffie–Hellman necesita una firma al lado?
- [ ] ¿Por qué RSA exige relleno?

Fuentes: tus presentaciones de clase (Chapter 1 – Introduction to Cryptography, Chapter 2 – Symmetric encryption, Chapter 3 – Asymmetric Encryption, Hash Functions and Digital Signature, y AppSec part 1). El esquema de Bitwarden, PBKDF2, HKDF, las cifras de iteraciones y la sección del KDF vienen de mi conocimiento general, no de las diapositivas: verifica los números con tus apuntes o con la documentación actual de Bitwarden y OWASP. Las diapositivas de AppSec y la parte final del capítulo 1 las leí completas; la hoja de capítulo 2 tiene una página más que no abrí.